

Git and GitHub Guide

Computer Science Study Notes, Part 2 of 6

1. What Git and GitHub Are

Git is a distributed version control system created by Linus Torvalds in 2005. It records the history of changes to a project, lets several people work on the same code, and makes it possible to return to any earlier version. Because Git is distributed, every developer has a full copy of the repository history on their own machine.

GitHub is a website that hosts Git repositories online and adds collaboration features such as pull requests, issues and project pages. Git and GitHub are not the same thing: Git is the tool that tracks versions, and GitHub is one of several services that store Git repositories remotely.

Git organizes work into three areas. The working directory holds the files you edit. The staging area holds the changes you have marked to include in the next commit. The repository stores the committed history.

2. The Basic Workflow

A typical first-time workflow uses these commands:

- `git init` creates a new local repository in the current folder.
- `git status` shows which files are modified, staged or untracked.
- `git add .` stages all changes in the current folder. To stage one file, use `git add filename`.
- `git commit -m "message"` saves the staged changes as a commit with a descriptive message.
- `git log --oneline` prints a compact list of previous commits.

A good commit message is short and describes what changed and why, for example `Add FastAPI health endpoint`. Commit small, logical units of work rather than one huge commit at the end of the project.

3. Branching and Merging

A branch is an independent line of development. It lets you build a feature without disturbing the stable code. The command `git branch feature-login` creates a branch, and `git switch feature-login` moves to it. The shortcut `git switch -c feature-login` creates and switches in one step. The older equivalent is `git checkout -b feature-login`.

When the feature is finished, switch back to the main branch and run `git merge feature-login`. If two branches changed the same lines of the same file, Git

cannot decide automatically and reports a merge conflict. You resolve it by editing the file to keep the correct content, removing the conflict markers, then staging the file and committing.

The command `git branch -M main` renames the current branch to main. Many tutorials use it before the first push, because GitHub uses main as the default branch name.

4. Working with Remote Repositories

A remote is a copy of the repository hosted elsewhere, such as on GitHub. To connect your local repository to a new empty GitHub repository, run `git remote add origin https://github.com/username/repo-name.git`. The name origin is the conventional name for the main remote. Then publish your commits with `git push -u origin main`. The option `-u` sets the upstream branch, so later you can simply run `git push`.

To copy an existing repository to your machine, use `git clone <url>`. To download new commits from the remote, use `git fetch`, which only downloads them, or `git pull`, which is a fetch followed by a merge into your current branch.

Before submitting a project, verify it like a stranger: clone your repository into a fresh folder and follow only the instructions in your README. If any step fails, fix the README.

5. The .gitignore File and Keeping Secrets Safe

A file named `.gitignore` lists patterns of files that Git must not track. Create it before your first commit. Typical entries for a Python project are `.venv/` (the virtual environment), `__pycache__` (compiled bytecode), `.env` (local settings and secrets), `*.log` (log files), and any very large data folders or model files.

Adding a pattern to `.gitignore` does not affect files that are already tracked. To stop tracking a file that was committed by mistake, run `git rm --cached filename` and then commit.

Never commit passwords, API keys or other secrets. If a secret was pushed by accident, treat it as compromised: revoke it and create a new one. Deleting it in a later commit is not enough, because it remains in the repository history. A common practice is to commit a `.env.example` file that lists variable names with safe placeholder values, while the real `.env` file stays ignored.